



Bureau d'étude d'Automatique

Mathis Magron

Noé Rigoletti

Année universitaire 2024/2025

Table des matières

1	Introduction	2
2	Le pendule inversé	2
2.1	Principe de la stabilisation par retour d'état	3
2.2	Contrôle par retour d'état pour le pendule inversé	3
2.3	Simulation du pendule inversé contrôlé par retour d'état et de sortie	4
2.3.1	Cas 1.1	5
2.3.2	Cas 1.2	5
2.3.3	Cas 1.3	5
2.3.4	Cas 1.4	6
2.3.5	Cas 1.5	7
2.3.6	Cas 1.6	7
2.4	Simulation du pendule inverse contrôle d'état en présence de capteur et prédicteur	7
2.4.1	Cas 1	8
2.4.2	Cas 2	9
2.4.3	Cas 3	9
3	Modèle continu et discret du robot Lego avec et sans contrôleur	10
3.1	Modèle continu et discret du robot Lego sans contrôleur	10
3.1.1	Cas1.1	11
3.1.2	Cas1.2	11
3.1.3	Cas1.3	12
3.2	Modèle continu et discret du robot Lego avec contrôleur et prédicteur/estimateur	12
3.2.1	Cas 1.1	13
3.2.2	Cas 1.2	14
3.2.3	Cas 1.3	14
3.3	Construction du modèle hybride	14
4	Pendule inversé sur Robot Lego NXT	16
5	Conclusion	17

1 Introduction

L'objectif final de ces séances de TP était de comprendre comment modéliser un robot Lego, afin de le maintenir à la verticale.

Pour ce faire, on a d'abord étudié un modèle simplifié, celui du pendule inversé contrôlé par retour d'état. Grâce à différentes simulations avec des valeurs initiales et des coefficients de contrôle (K) variables. Par la suite, on modélise alors, toujours par le biais de Simulink le robot Lego pendule inversé, modèle qu'on réutilisera lors de l'implémentation du vrai robot.

Après cela, on pouvait alors implémenter un programme dans le robot, prenant en compte le retour d'état, pour le stabiliser même en présence d'une perturbation. Le robot tient finalement en équilibre après étalonnage, et dans la limite d'une perturbation légère.

2 Le pendule inversé

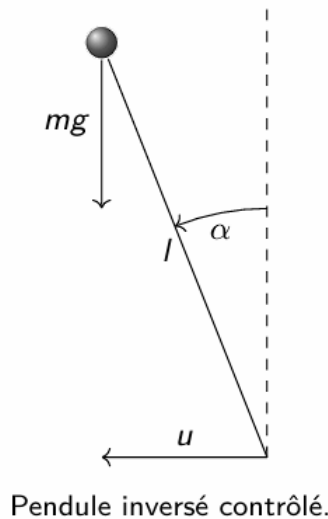


FIGURE 1 –

Lors des premiers TP, le but était de modéliser le comportement d'un pendule inversé contrôlé par retour d'état. Le principe fondamental de la dynamique appliqué à notre système, en considérant le problème sans frottement, nous donne les équations suivantes :

On pose

$$x(t) = (\alpha(t), \dot{\alpha}(t))$$

$$\begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = \frac{g}{l} \sin(x_1(t)) - \frac{\cos(x_1(t))u(t)}{l} \\ x_1(0) = x_{0,1} = \alpha_0 \\ x_2(0) = x_{0,2} = \dot{\alpha}_0 \end{cases}$$

Les notations mathématiques sont données comme suit :

$g = 9.81$, l'accélération de la pesanteur.

$l = 10$, la longueur du pendule.

$t_0 = 0$, l'instant initial.

$x_e = (0, 0)$, point d'équilibre du système.

$u_e = 0$,

$u(t) = u_e + K(x(t) - x_e)$, contrôle par retour d'état sur l'accélération du déplacement que l'on peut exercer horizontalement.

$K = (k_1, k_2)$.

Ici k_1 et k_2 représentent la quantité d'énergie que l'on peut injecter dans le système. Plus ils sont grands, plus le moteur peut fournir de l'énergie au système.

2.1 Principe de la stabilisation par retour d'état

Ici, le pendule inversé contrôlé est un système non linéaire autonome :

$$\dot{x}(t) = f(x(t), u(t))$$

et on s'intéresse à la stabilisation au tour du point de fonctionnement (x_e, u_e) par retour d'état linéaire (au sens large, c'est-à-dire linéaire ou affine) :

$$u(t) = u_e + K(x(t) - x_e) \quad (1)$$

Le système bouclé est donc

$$\dot{x}(t) = f(x(t), u_e + K(x(t) - x_e)) =: g(x(t))$$

On remarque alors que x_e est un point d'équilibre de g , c'est à dire $g(x_e) = 0$. La stabilité asymptotique de x_e est liée aux valeurs propres de $g'(x_e)$. D'où :

$$g'(x) = \frac{\partial f}{\partial x}(x_e, u_e) + \frac{\partial f}{\partial u}(x_e, u_e)K =: A + BK. \quad (2)$$

En conclusion, il faut trouver $K \in M_{n,m}(\mathbb{R})$ telle que les valeurs propres de

$$g'(x_e) = A + BK$$

soient à partie réelle strictement négative.

2.2 Contrôle par retour d'état pour le pendule inversé

Tout d'abord regardons si l'origine est asymptotiquement stable pour le système non contrôlé :

$$\dot{x}(t) = \begin{pmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \end{pmatrix} = \begin{pmatrix} x_2(t) \\ \frac{g}{l} \sin(x_1(t)) \end{pmatrix}$$

Au voisinage de 0 : $\sin(x_1) \sim x_1$ et $\cos(x_1) \sim 1$. Donc

$$\dot{x}(t) = \begin{pmatrix} x_2 \\ \frac{g}{l} x_1 \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & 1 \\ \frac{g}{l} & 0 \end{pmatrix}}_{=A} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

D'où

$$\dot{X}(t) = AX(t)$$

Notons λ_1 et λ_2 les valeurs propres de A . On obtient :

$$\begin{cases} \lambda_1 + \lambda_2 = 0 = \text{tr}(A) \\ \lambda_1 \lambda_2 = -\frac{g}{l} = \det(A) \end{cases}$$

Soit

$$\begin{cases} \lambda_1 = \sqrt{\frac{g}{l}} > 0 \\ \lambda_2 = -\sqrt{\frac{g}{l}} < 0 \end{cases}$$

$\lambda_1 > 0$ et $\lambda_2 < 0$ donc le système n'est pas asymptotiquement stable à l'origine.

Maintenant, on cherche une condition sur K pour que le contrôle par retour d'état $u(t) = u_e + K(x(t) - x_e)$ stabilise asymptotiquement le système en un point de fonctionnement tel que $\cos x_{e,1} > 0$.

Après calcul, on obtient que les points de fonctionnement du système (x_e, u_e) sont de la forme :

$$(x_e, u_e) = (x_1, 0, g \tan(x_1)) \text{ avec } x_1 \neq \frac{\pi}{2} + K\pi$$

Ce qui nous donne finalement les conditions suivantes :

$$K = (k_1, k_2)$$

avec $k_1 > g$ et $k_2 > 0$

2.3 Simulation du pendule inversé contrôlé par retour d'état et de sortie

Pour contrôler asymptotiquement le système, si $(\alpha_0, \dot{\alpha}_0)$ est suffisamment proche de x_e , il suffit que

- $k_1 > g$,
- $k_2 > 0$.

Nous avons commencé par réaliser des tests succincts, avec différentes valeurs en entrée, des méthodes d'intégration différentes, et enfin des temps de simulation variés.

Cas	x_0	t_f	K	Méthode
Cas 1.1	$(\frac{\pi}{20}, 0)$	10	(30, 10)	ode45
Cas 1.2	$(\frac{\pi}{20}, 0)$	100	(10, 1)	ode45
Cas 1.3	$(\frac{\pi}{20}, 0)$	100	(10, 1)	Euler, ode1
Cas 1.4	$(\frac{\pi}{20}, 0)$	1000	(10, 1)	Euler, ode1
Cas 1.5	$(\frac{\pi}{10}, 0)$	100	(10, 1)	ode45
Cas 1.6	$(\frac{\pi}{10}, 0)$	100	(30, 10)	ode45

TABLE 1 – Tableau des différents cas de simulation

2.3.1 Cas 1.1

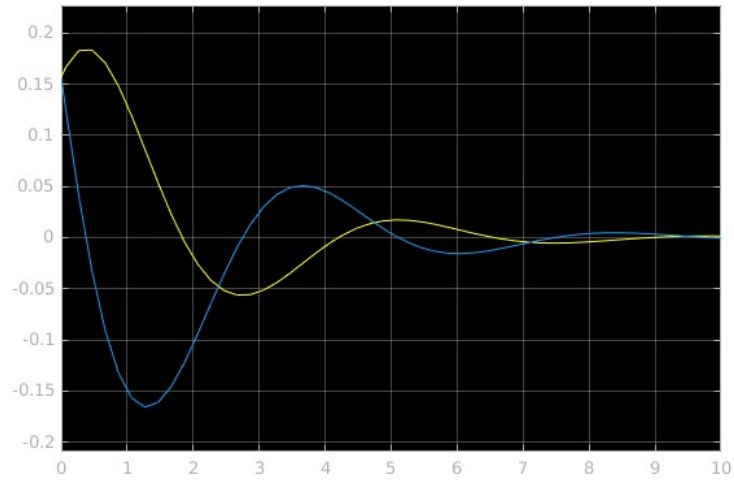


FIGURE 2 – Cas 1.1

Dans cette simulation, on se rend compte que les paramètres du système convergent vers un état d'équilibre. Le temps de stabilisation concorde avec le temps de la simulation, qui est de 10 secondes. Le contrôle est donc suffisamment puissant pour atteindre l'équilibre assez rapidement.

2.3.2 Cas 1.2

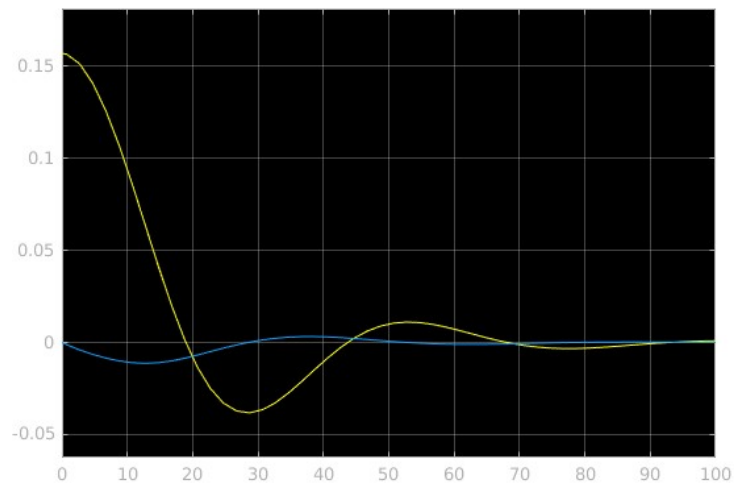


FIGURE 3 – Cas 1.2

Ici, le contrôle ne parvient pas à stabiliser le système aussi rapidement que pour le premier cas, dû au fait que l'on a réduit la valeur de K (cela revient à baisser l'intensité du moteur dans notre robot).

2.3.3 Cas 1.3

Les résultats sont sensiblement les mêmes, cependant en utilisant la méthode d'intégration d'Euler, on engendre plus d'erreurs et le système converge encore moins rapidement, pour des mêmes valeurs en entrée.

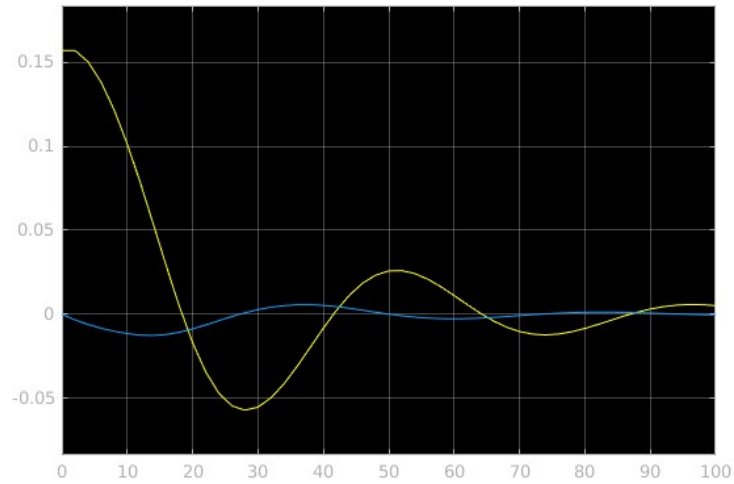


FIGURE 4 – Cas 1.3

2.3.4 Cas 1.4

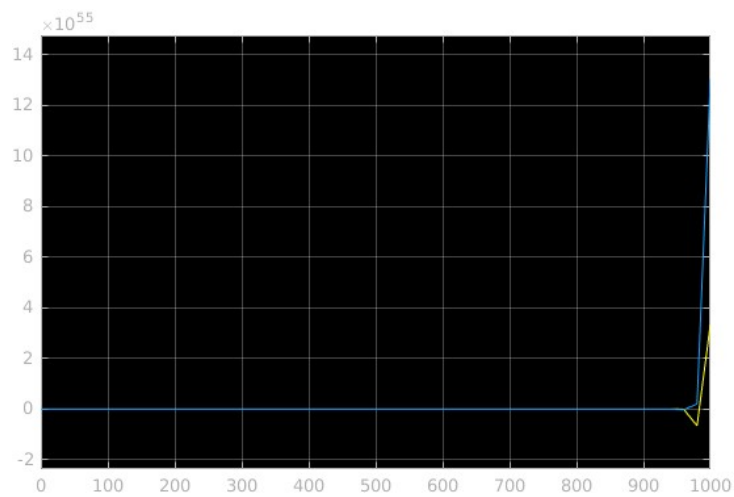


FIGURE 5 – Cas 1.4

En utilisant les mêmes données et contrôle que la simulation précédente, avec un temps de fonctionnement plus long (1000s), on observe que la méthode d'Euler accumule les erreurs, car il est difficile de modéliser informatiquement des systèmes réels. On préférera l'utilisation de la méthode de Runge-Kutta aux temps longs.

2.3.5 Cas 1.5

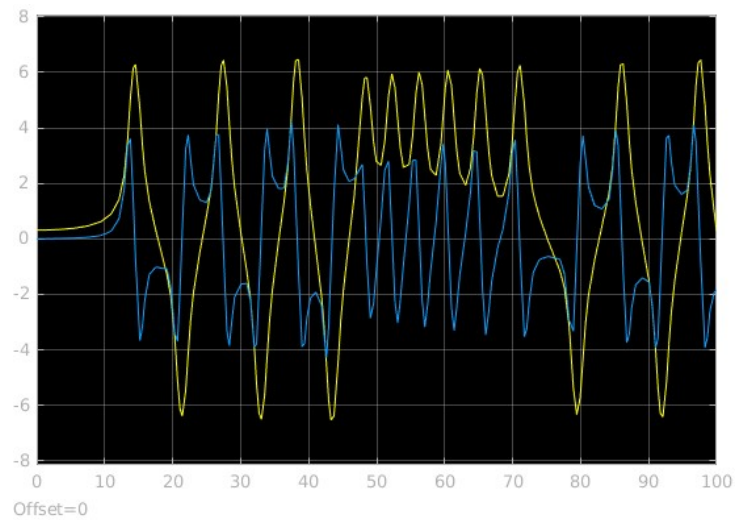


FIGURE 6 – Cas 1.5

En augmentant la valeur initiale de l'angle du pendule, le contrôle n'est plus capable de stabiliser le système. En analysant les courbes, on se rend bien compte que le pendule va osciller autour de la position d'équilibre, sans jamais réellement l'atteindre. On peut imaginer que cela est dû au trop peu d'énergie fourni par le moteur pour tenter de redresser le pendule.

2.3.6 Cas 1.6

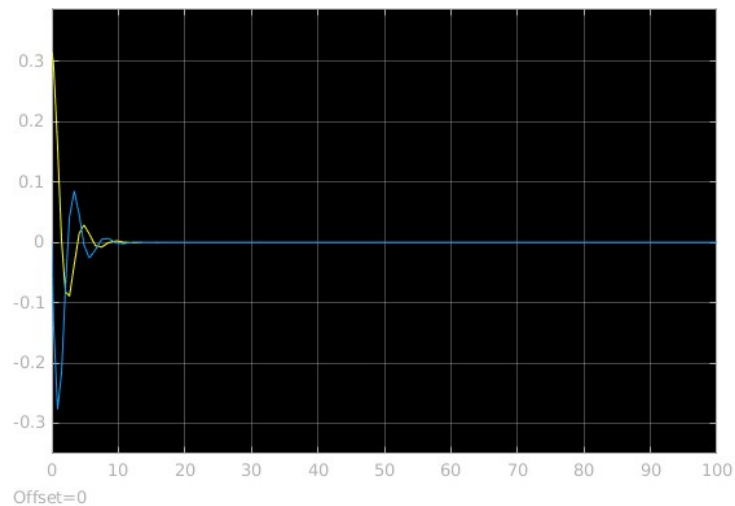


FIGURE 7 – Cas 1.6

On confirme l'hypothèse précédente en augmentant la valeur de K , toujours avec l'angle initial élevé. Cette fois-ci, le système se stabilise au bout de 10 secondes.

2.4 Simulation du pendule inverse contrôle d'état en présence de capteur et prédicteur

Dans cette partie, le but est toujours de modéliser le système du pendule inversé. Cependant, on va supposer que l'on ne peut pas mesurer en sortie l'angle α , mais seulement la vitesse

angulaire de rotation $\dot{\alpha}$. Cette situation va nous permettre de nous rapprocher du modèle d'un cas réel, où toutes les variables ne sont pas accessibles de par le nombre limité de capteurs dans un système.

Pour palier à ce manque d'informations, une des solutions est d'utiliser un prédicteur, composant capable d'estimer et donc reconstruire une variable d'état (ici α) à partir des autres (ici $\dot{\alpha}$).

Ici, c'est un intégrateur continu qui va permettre de calculer en temps réel la valeur de α , par le calcul suivant :

$$\alpha(t) = \alpha(0) + \int_0^t \dot{\alpha}(\tau) d\tau$$

Grâce à la même méthode que précédemment, on va réaliser plusieurs simulations, pour observer le comportement du système dans différents cas :

Cas	x_0	t_f	K	Pas	Méthode
Cas 1	$(\frac{\pi}{20}, 0)$	100	(10, 1)	Par défaut	ode45
Cas 2	$(\frac{\pi}{20}, 0)$	100	(10, 1)	0.001	ode45
Cas 3	$(\frac{\pi}{20}, 0)$	100	(10, 1)	5	Euler, ode1

TABLE 2 – Tableau des différents cas de simulation

2.4.1 Cas 1

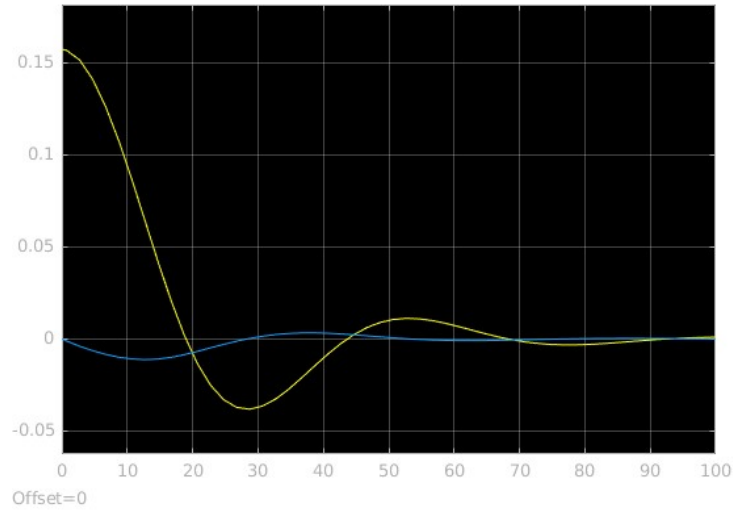


FIGURE 8 – Cas 1

Ce cas est très similaire au Cas 1.2, avec pour seule différence la présence du prédicteur pour estimer la valeur de α . Le choix des variables et de la méthode d'intégration est donc pertinent.

2.4.2 Cas 2

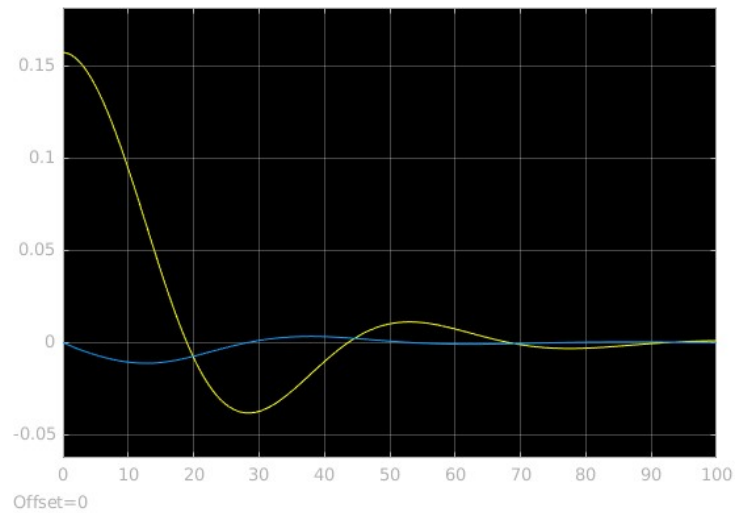


FIGURE 9 – Cas 2

Cette fois-ci, on a juste modifié la valeur du pas de temps. On obtient donc plus de points, mais cela n'influe pas sur la contrôlabilité du système. on peut alors se dire que le premier cas est tout aussi pertinent puisque qu'il n'omet pas d'informations utiles. En effet les deux premiers cas ont des pas de temps suffisamment petits devant la constante de temps caractéristique du système.

2.4.3 Cas 3

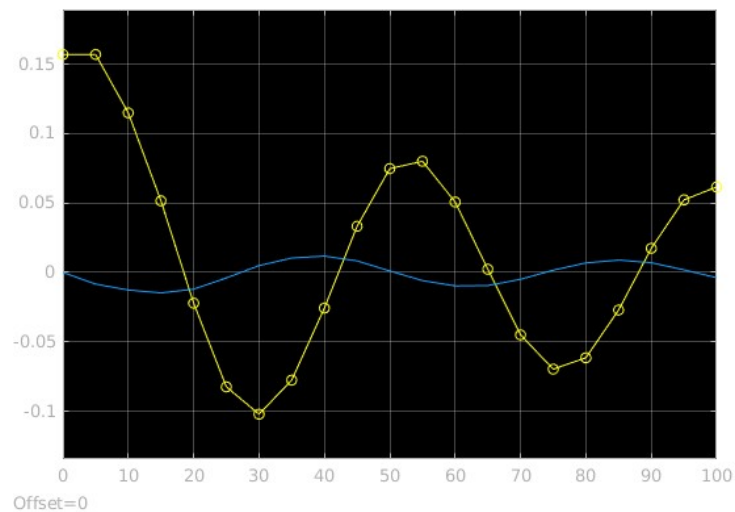


FIGURE 10 – Cas 3

Dans cette dernière simulation, on augmente le pas de temps à 5 secondes, et on opte pour la méthode d'intégration d'Euler. On observe bien que le système n'est pas asymptotiquement stable, bien qu'on ait choisi un grand temps de simulation. Cela est dû au fait que le pas de temps est trop grand devant le temps caractéristique du système. On va donc préférer un pas de temps similaire aux deux premiers cas lors de la création de notre modèle.

3 Modèle continu et discret du robot Lego avec et sans contrôleur

On va maintenant simuler le comportement du robot Lego, en se servant des modélisations faites dans le TP2. Cette fois-ci, la variable d'état est dans R^4 . En effet en tout temps t l'état du système est décrit par $(\theta, \psi, \dot{\theta}, \dot{\psi})$. Pour cela, on crée un modèle sur Simulink pour observer le comportement asymptotique du système. On donne ici par exemple le modèle avec capteur.

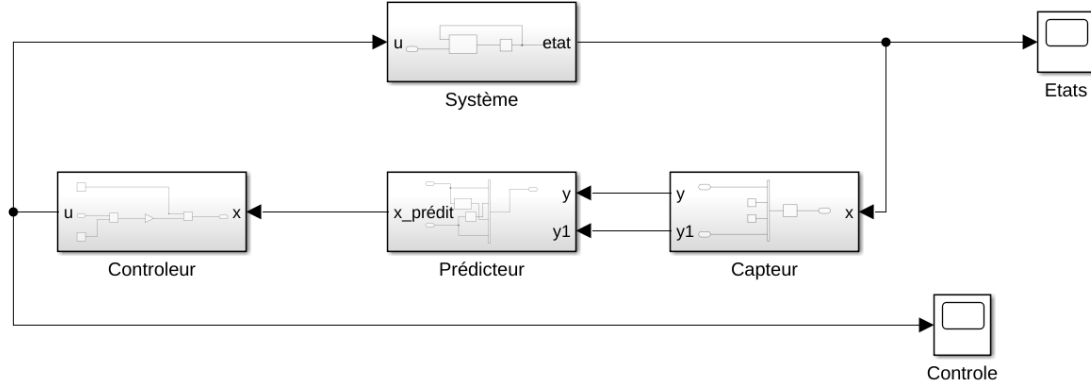


FIGURE 11 – Modèle Simulink du robot Lego

3.1 Modèle continu et discret du robot Lego sans contrôleur

Par calcul, on retrouve la valeur des coefficients de K . Ici on a donc $K = (0.6700, 19.9055, 1.0747, 1.9614)$.

On va donc pouvoir par la suite réaliser de nouvelles simulations, en changeant les paramètres x_0 ou encore la méthode d'intégration. Ce premier jeu de test est réalisé sur un modèle sans capteur. Le tableau suivant récapitule les tests réalisés.

Cas	x_0	t_f	K	Méthode
Cas 1.1	$(0,0,0,0)$	2	$(10, 1)$	ode45
Cas 1.2	$(0, \frac{\pi}{5}, 0, 0)$	5	$(10, 1)$	ode45
Cas 1.3	$(0, \frac{\pi}{10}, 0, 0)$	5	$(10, 1)$	ode45

TABLE 3 – Tableau des différents cas de simulation

Sur les simulations suivantes, les quatre courbes représentent l'évolution temporelle des 4 valeurs d'état, dont on suppose qu'on obtient la valeur à chaque instant par des capteurs.

3.1.1 Cas1.1

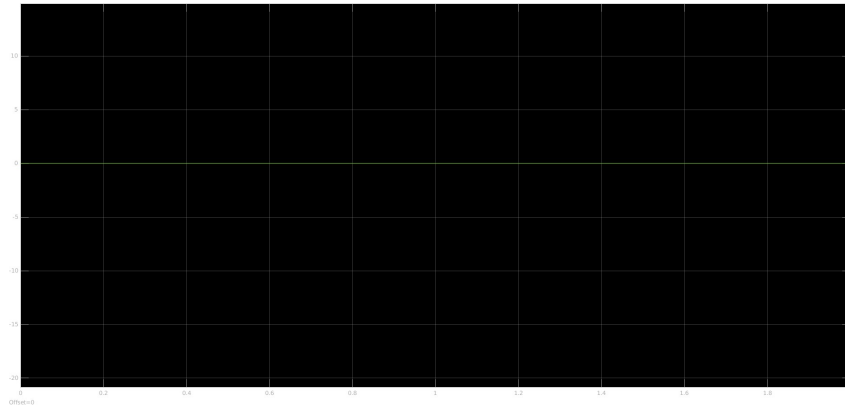


FIGURE 12 – Cas 1.1

En simulant les variables d'état du système, on observe qu'elles sont constantes et valent 0. Cela est dû au fait que les variables initiales sont nulles ($x_0 = (0, 0, 0, 0)$). Le système est donc déjà à la position d'équilibre et aucune correction n'a besoin d'être faite.

3.1.2 Cas1.2

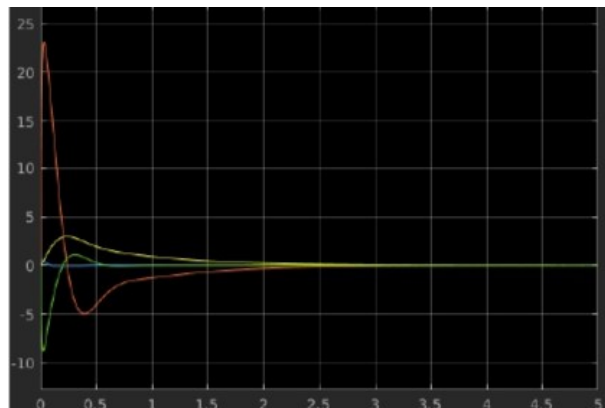


FIGURE 13 – Cas 1.2

Ici la vitesse angulaire (courbe en rouge) est très grande devant les autres variables. Cela s'explique par une nécessité de redresser rapidement le système pour une position déséquilibrée. Finalement, le système semble atteindre un équilibre stable assez rapidement (inférieur à 5 secondes).

3.1.3 Cas1.3

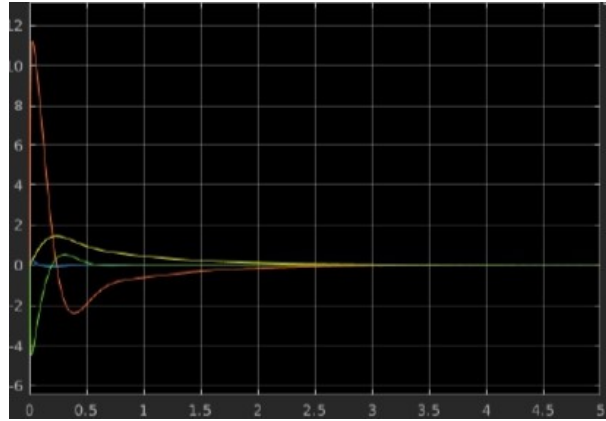


FIGURE 14 – Cas 1.3

L'allure des courbes dans ce cas est très similaire à celle du cas précédent. La différence d'amplitude, elle, est liée à la différence d'angle initial sur θ . En effet ici on a $\theta = \frac{\pi}{10}$ soit deux fois plus petit que dans le Cas 1.2. Logiquement, la vitesse angulaire est moins élevée puisque le système est déjà plus proche d'un équilibre.

3.2 Modèle continu et discret du robot Lego avec contrôleur et prédicteur/estimateur

On va maintenant simuler le comportement du robot en situation réelle, c'est-à-dire avec des prédicteurs capables d'estimer la valeur des variables d'état manquantes.

En effet le système est composé de deux capteurs :

- Un gyroscope mesurant la vitesse de changement d'angle du corps du robot ($\dot{\psi}$).
- Un capteur mesurant l'angle du corps du robot par rapport à la verticale (θ).

Un prédicteur doit donc en temps réel calculer la valeur de $\dot{\theta}$ et ψ par la même méthode qu'expliquée précédemment :

$$\forall t \in [0; t_f] :$$

$$\psi(t) = \psi(0) + \int_0^t \dot{\psi}(\tau) d\tau$$

Et

$$\dot{\theta}(t) = \frac{d}{dt}\theta(t)$$

On inclue alors un le prédicteur suivant dans notre modèle Simulink :

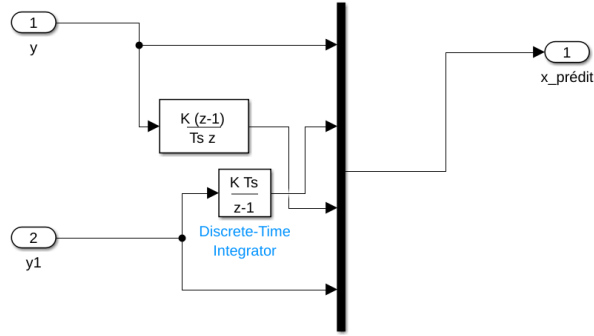


FIGURE 15 – Modèle du prédicteur

En se fixant le même jeu de tests, on va de nouveau modéliser le comportement du système dans plusieurs situations en prenant en compte les prédicteurs.

3.2.1 Cas 1.1

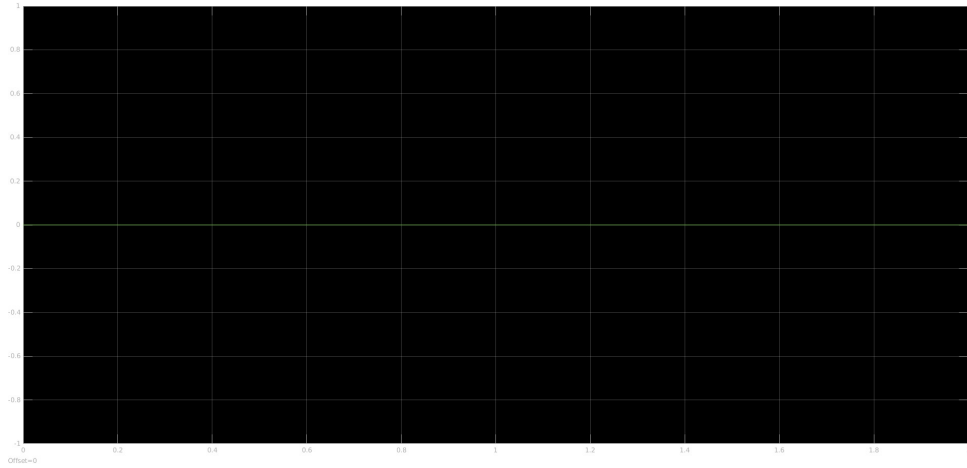


FIGURE 16 – Cas 1.1

Comme pour la simulation sans prédicteur, en partant de $x_0 = (0, 0, 0, 0)$, le système étant déjà stabilisé, il reste à l'équilibre $\forall t > 0$.

3.2.2 Cas 1.2

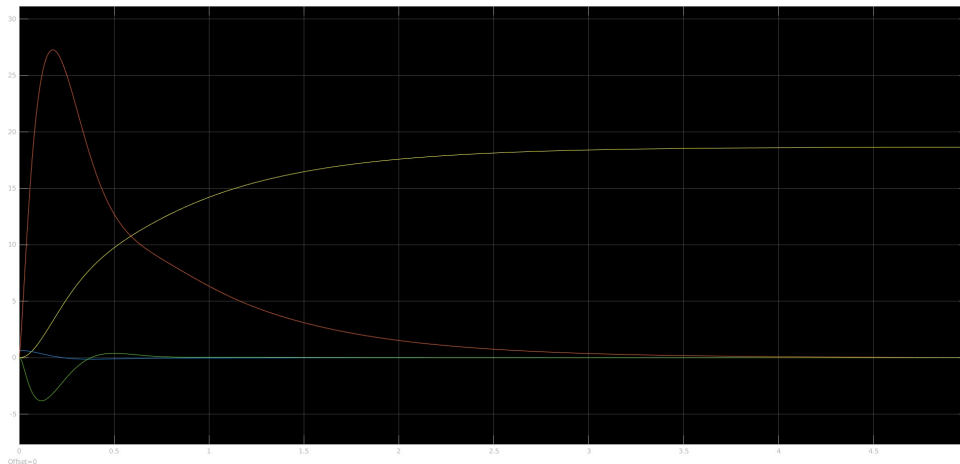


FIGURE 17 – Cas 1.2

Ici, le système se stabilise très rapidement autour d'une position d'équilibre. Cependant on remarque que la valeur de θ (courbe en jaune) ne converge pas vers le même équilibre que précédemment (valeur non nulle). On peut en déduire que le prédicteur va influencer sur la convergence du système, puisque le calcul de ψ et $\dot{\theta}$.

3.2.3 Cas 1.3

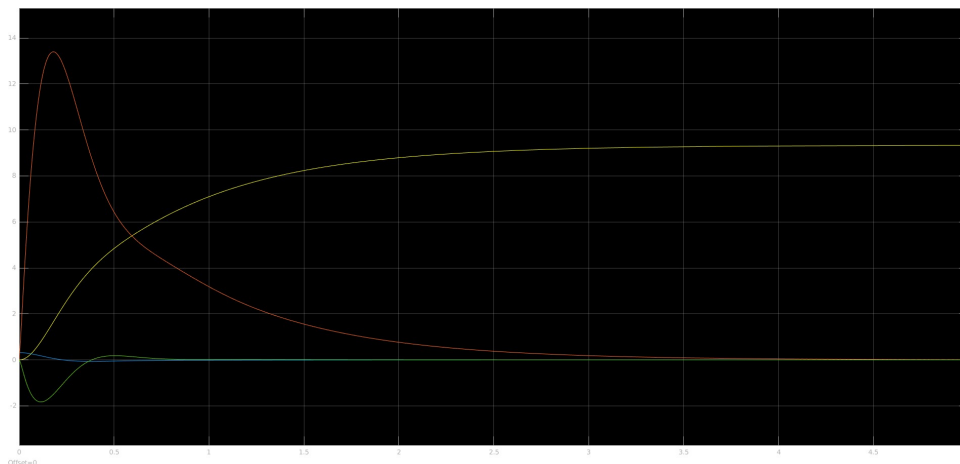


FIGURE 18 – Cas 1.3

De la même manière que sans le prédicteur, en posant $x_0 = (0, \frac{p^i}{10})$ on diminue la valeur initiale de l'angle θ , et on réduit l'écart entre cette position et la position d'équilibre. Il n'est donc pas anormal d'observer des amplitudes moins fortes quant aux évolutions temporelles des grandeurs d'état.

3.3 Construction du modèle hybride

Dans la suite, on va donc implanter le prédicteur et le contrôleur sous la forme d'un modèle discret. Cependant le modèle physique associé aux équations régissant l'équilibre du système étant continu, il est nécessaire d'ajouter un moyen de convertir les données d'entrée dans le

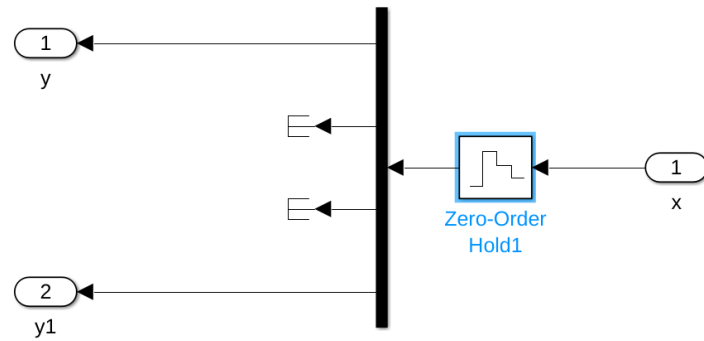


FIGURE 19 – Capteur avec sortie discrète

capteur en les discrétisant. On utilise un bloc Zero-Order Hold permettant d'effectuer cette opération. Le capteur devient alors :

On parvient donc à tracer l'évolution du système réel :

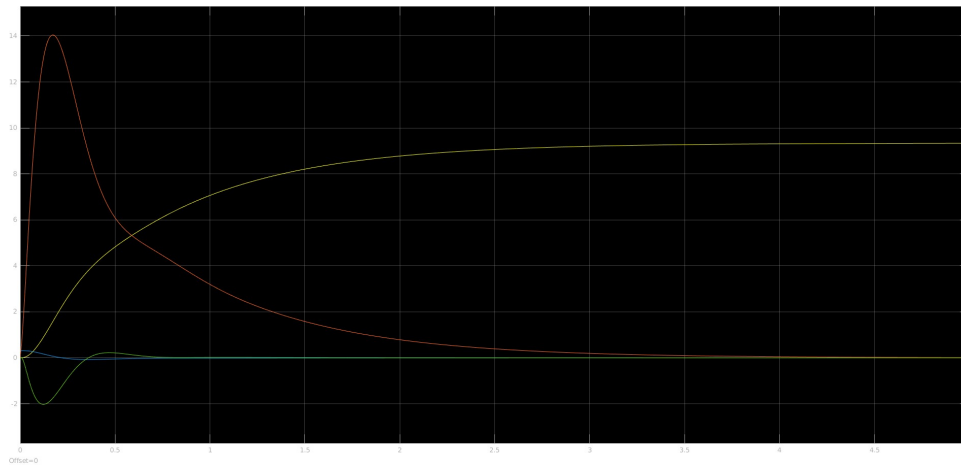


FIGURE 20 – Modèle Hybride

Les courbes se rapprochent beaucoup des précédentes, on en conclue que la discrétisation des données est très pertinente, et nous amène à un équilibre asymptotique.

4 Pendule inversé sur Robot Lego NXT

Dans un dernier temps, nous synthétisons les connaissances acquises lors des TP précédents afin de faire tenir en équilibre un système réel : le Robot Lego NXT. En implémentant différents modes de fonctionnement dans le logiciel du robot, on peut alors initialiser le robot, calibrer son gyroscope, et le faire fonctionner.

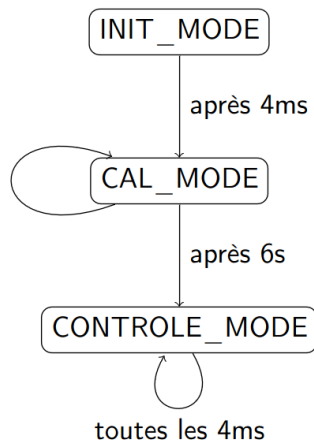


FIGURE 21 – Modes de fonctionnement

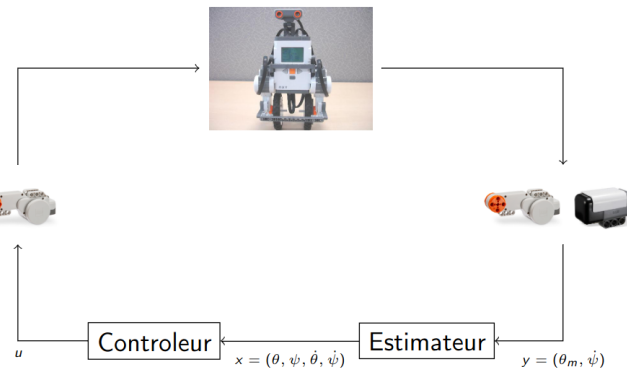


FIGURE 22 – Système réel du robot

Nous nous sommes occupés uniquement du mode *CONTROLE_MODE*. On a implémenté le code suivant dans le robot :

```

41  float K[4]={0.67,19.9053,1.0747,1.9614};

98  case(CONTROL_MODE):
99
100     dt = delta_t();
101
102     y[0]= getMotorAngle();
103     y[1]= getGyro(gyro_offset);
104     x[2]= (y[0]+x[1]-x[0])/dt;
105     x[0]= y[0]+x[1];
106     x[1]= x[1]+y[1]*dt;
107     x[3]= y[1];
108
109     u = ue;
110     for (int i = 0; i<4; i++){
111         u=u+ K[i]*(x[i]-xe[i]);
112     }
113     nxt_motors_set_command(u);
114     break;
115

```

FIGURE 23 – Code de *CONTROLE_MODE*

Finalement, après allumage et initialisation du robot, il parvient à s'équilibrer, et à réagir inversement aux perturbations pour toujours parvenir à un état d'équilibre. Il existe quand même des limites à l'utilisation, par exemple lorsque la perturbation est trop forte, ou que la

surface disponible pour le robot n'est pas assez grande pour l'équilibrage. Cela est dû à la valeur trop faible du contrôle K , équivalente à la quantité d'énergie qu'il est possible d'injecter dans le système via les moteurs.

5 Conclusion

Au cours de ce projet, nous avons mis en pratique des concepts fondamentaux de l'automatique appliqués à un système complexe : le pendule inversé et sur le robot Lego NXT. Nous avons modéliser, simuler et stabiliser le pendule inversé grâce à différentes approches, allant du contrôle par retour d'état aux modèles hybrides intégrant capteurs et prédicteurs.

Ces expériences ont mis en évidence l'importance d'un choix judicieux des paramètres, notamment les gains du contrôle K , pour garantir la stabilité du système tout en tenant compte des limitations physiques et des perturbations extérieures. Les simulations ont également mis en évidence l'impact des méthodes d'intégration numérique et du pas de temps sur la précision et la convergence des résultats.

Enfin, l'application au robot réel a confirmé la validité des modèles, tout en révélant certaines contraintes pratiques, telles que la sensibilité aux perturbations importantes.

Ainsi, ce projet a permis de renforcer notre compréhension des systèmes dynamiques et leur contrôle et faire un lien solide entre théorie et pratique.